

ESOF 322

Software Engineering

LECTURE: 10

J. L. Pach

OUTLINE PR

- Project Backlog
- Semantic Versioning
- Code Merge Strategy Guide

PROJECT

Backlog

MONTANA TECH MINER - PROJECT BACKLOG

Current (DONE)

- ⚒ v0.1.0 Player movement + collision
- ⚒ v0.2.0 Tilemap + grid system
- ⚒ v0.2.5 Code Review + PR workflow training

UPCOMING SPRINTS (NO BLOCKING DEPENDENCIES)

v0.4.0 Stamina System Logic + Debug Output

- Jump costs stamina
- Food regenerates stamina
- Movement/jump penalties when stamina is low
- Debug text output

v0.4.1 Stamina Bar Visualization

- Blue stamina bar on HUD
- Synchronized with v0.4.0 logic

SEMANTIC

Versioning

SEMANTIC VERSIONING GUIDE

Overview

We use **Semantic Versioning** (SemVer) to track releases of the Montana Tech Miner project. This helps us communicate what changed and how significant the changes are.

```
v0.4.1
 ↓ ↓ ↓
 | | └── PATCH: Bug fixes, small improvements
 | └──── MINOR: New features, backwards compatible
 └────── MAJOR: Breaking changes, major milestones
```

OUR VERSIONING SCHEME

`v0.x.x` = Pre-Release / Alpha Phase

Since the game is under active development, all versions start with `0`. This means:

- ⚒ Features may change or be removed.
- ⚒ We're still experimenting with design.
- ⚒ Breaking changes are acceptable in v0.x releases.

`v1.0.0` = First Stable Release

When we have a complete, playable game with all core features, we release `v1.0.0`.

EXAMPLES FROM MONTANA TECH MINER

Current Release History

```
v0.1.0 → Player movement + basic collision
v0.2.0 → Tilemap system + grid
v0.2.5 → Code Review workflow training (minor polish)
v0.3.0 → Render target scaling (if implemented)
v0.4.0 → Stamina system logic + debug output
v0.4.1 → Stamina bar visualization
v0.5.0 → Food pickup mechanic
v0.6.0 → Mining/digging system
v0.6.5 → New tile types (STONE, COAL, CRYSTAL)
v0.7.0 → Health system
v0.8.0 → Win/lose conditions
v0.9.0 → Polish (graphics, sounds, menu)
v1.0.0 → First stable release
```

Rules for Version Bumping 1/2

PATCH (X.X.Z)

- Bug fixes in existing features
- Small optimizations
- Documentation updates
- No new functionality

Example:

- v0.4.0 → v0.4.1: Fix stamina bar alignment bug

MINOR (X.Y.X)

- New features added
- New tiles, new mechanics
- Backwards compatible with previous v0.x releases
- Existing code still works

Example:

- v0.4.0 → v0.5.0: Add food pickup mechanic (new feature)
- v0.5.0 → v0.6.0: Add mining system (new feature)

RULES FOR VERSION BUMPING 2/2

MAJOR (X.x.x)

- Significant overhaul
- Breaking changes to existing systems
- Game is now "complete enough" to move to v1.0.0

Example:

- v0.9.0 → v1.0.0: Game is complete and stable

HOW TO CREATE A RELEASE TAG IN GIT

1. Ensure code is on main branch

```
git checkout main
git pull origin main
```

2. Create an annotated tag

```
git tag -a v0.4.1 -m "Release: Stamina Bar Visualization"
```

3. Push tag to GitHub

```
git push origin v0.4.1
```

Or push all tags at once:

```
git push origin --tags
```

4. Verify tags on GitHub

```
Go to your repository → Releases tab. You should see the new tag listed.
```

CODE

Merge Strategy Guide

Overview

When multiple students implement the same feature (like Stamina System), we need a **merge strategy** to decide which code goes into `main` and how. This guide explains strategies used in modern software development.

Context: Why We Need a Strategy

The Problem:

- ⚒ 4 students in Team Alpha all implement v0.4.0 (Stamina System).
- ⚒ All 4 PRs work, but with slightly different approaches.
- ⚒ All 4 PRs pass code review.
- ⚒ We can't merge all 4 — that would create duplicate code.

The Question:

How do we choose which code wins?

Strategy 1: First One Wins (Simplest)

HOW IT WORKS

```
Timeline:
PR #10 (Mario) → Code review → Approved → MERGED to main
PR #11 (Luigi) → Code review → Approved → CLOSED (not merged)
PR #12 (Peach) → Code review → Approved → CLOSED (not merged)
PR #13 (Toad) → Code review → Approved → CLOSED (not merged)
```

PROS

- Simple, fast
- No conflicts
- Clear winner

CONS

- Unfair to other students (their work is discarded)
- You may lose good ideas from PR `#11`, `#12`, `#13`
- Demoralizing for students who wrote better code but were second

WHEN TO USE

- When time is critical
- When one PR is objectively superior
- In early development sprints

Example Command

```
git checkout main
git pull origin main
git merge --squash alpha/feature/stamina_system/mario
git commit -m "feat: Stamina system (v0.4.0) - mario"
git push origin main
git tag -a v0.4.0 -m "Stamina system complete"
```

Strategy 2: Cherry-Picking

HOW IT WORKS

Manually review all 4 PRs and **extract the best parts from each**, combining them into one optimal implementation.

PR #10 (mario):
✓ Good stamina drain logic
✓ Good variable naming
✗ Poor stamina regeneration

PR #11 (luigi):
✓ Excellent regeneration logic
✓ Clear comments
✗ Messy variable names

PR #12 (peach):
✓ Best debug output
✓ Good error handling
✓ Clean code structure

PR #13 (toad):
✗ Doesn't compile
✗ Incomplete

Decision:

- ⚒ Take drain logic from mario
- ⚒ Take regen logic from luigi
- ⚒ Take debug output from peach
- ⚒ Take error handling from peach
- ⚒ Create one clean, synthesized version

Implementation Steps

```
# 1. Create a new branch for synthesis
git checkout main
git pull origin main
git checkout -b feat/stamina_synthesis

# 2. Cherry-pick specific commits from mario
git cherry-pick mario_commit_hash_1
git cherry-pick mario_commit_hash_2

# 3. Cherry-pick specific commits from luigi
git cherry-pick luigi_commit_hash_1

# 4. Manually edit to combine best parts
# (Edit files to blend the approaches)

# 5. Create one clean synthesized commit
git add .
git commit -m "feat: Stamina system (v0.4.0) - Synthesized from team review
```

Combines:

- ⚒ Drain logic from PR `#10` (Mario)
- ⚒ Regen logic from PR `#11` (Luigi)
- ⚒ Debug output from PR `#12` (Peach)

Team consensus decision made during code review session."

Merge to main

```
git checkout main
git merge --squash feat/stamina_synthesis
git push origin main
git tag -a v0.4.0 -m "Stamina system complete"
```

PROS

- **Fair:** Acknowledges good work in all PRs
- **Quality:** Final code is the best of all implementations
- **Educational:** Students learn from each other's approaches
- **Team morale:** Everyone contributed to final product

CONS

- Time-consuming (requires careful code review and synthesis)
- Risk of introducing bugs during synthesis
- Requires deep understanding of all 4 implementations

WHEN TO USE

- When all PRs are good quality but slightly different
- In an educational setting (like this course)
- When combining different approaches makes sense

EXAMPLE: COMPARISON TABLE

Criterion	mario	luigi	peach	toad	CHOOSE
Drain logic	✓✓	✓	✓	✗	mario
Regen logic	✓	✓✓✓	✓	✗	luigi
Debug output	✓	✓	✓✓✓	✗	peach
Error handling	✓	✓	✓✓	✗	peach
Variable naming	✓✓	✓	✓✓	✗	mario
Code comments	✓	✓✓	✓	✗	luigi
Compiles	YES	YES	YES	NO	All but toad

Strategy 3: Squash and Merge (Git Technique)

HOW IT WORKS

When merging a PR to `main`, combine all commits from the PR into **one single commit**.

Before (PR has 5 commits):

```
ommit abc123: Add stamina variable
commit def456: Add drain logic
commit ghi789: Fix bug in regen
commit jkl012: Add debug output
commit mno345: Final polish
```

After squash merge to main:

```
commit xyz999: feat: Stamina system (v0.4.0)
  (contains all changes from abc123-mno345)
```

WHY USE IT

- Keeps `main` branch history **clean and readable**
- Each PR = one logical commit
- Easier to revert a feature (just revert one commit)
- Easier to cherry-pick a feature (it's all in one commit)

Git Command

```
git checkout main
git pull origin main

# Merge with --squash flag
git merge --squash feature/alpha/stamina_system/mario

# Create one commit
git commit -m "feat: Stamina system (v0.4.0) - mario
```

Implements:

- Stamina drain on jump
- Stamina regeneration from food
- Movement/jump penalties when low"

```
git push origin main
```

PROS

- Clean history on `main`
- Easy to track logical changes
- PR still visible in GitHub (even after merge)

CONS

- Loses individual commit history
- Can't see step-by-step development

WHEN TO USE

- Always (recommended for most projects)
- Especially when `main` branch is critical

Strategy 4: Rebase and Merge

HOW IT WORKS

Instead of creating a merge commit, **replay all PR commits on top of main**.

Before:

```
main: A ← B ← C
PR: D ← E ← F
```

After (rebase):

```
main: A ← B ← C ← D ← E ← F
```

PROS

- Linear, clean history (no merge commits)
- Easier to bisect (find which commit broke something)
- Each commit is testable independently

CONS

- Rewrites commit history (destructive)
- Can cause conflicts if not careful
- Less clear that changes were reviewed as a unit

Git Command

```
git checkout feature/alpha/stamina_system/mario
git rebase origin/main

git checkout main
git merge --ff-only feature/alpha/stamina_system/mario
git push origin main
```

WHEN TO USE

- In projects with linear development workflows
- When individual commits are well-tested
- Less common in educational settings

THANK

You